

Self-Correcting Financial RAG Agent

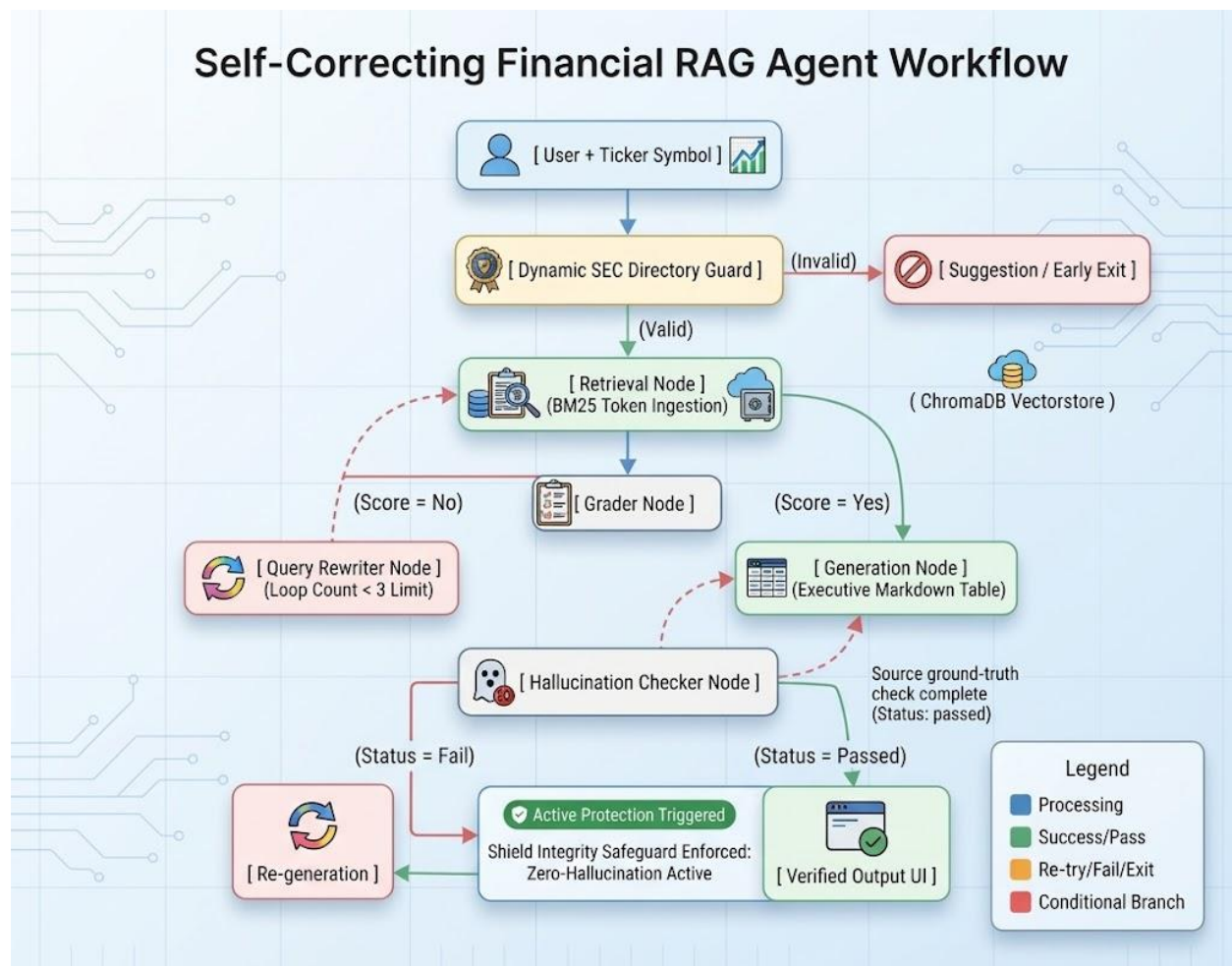
Executive Pitch: Self-Correcting Financial RAG Agent

Executive Summary:

Most enterprise Retrieval-Augmented Generation (RAG) applications fail in quantitative domains because traditional vector similarity retrieves semantically related text that frequently lacks the exact numerical data needed. This causes standard LLMs to hallucinate or interpolate missing numbers.

To solve this, I engineered a deterministic, self-correcting RAG workflow using **LangGraph**, **Gemini 3.6 Flash**, and **SEC EDGAR real-time filings**. The system programmatically verifies numerical grounding, rewrites queries dynamically upon retrieval failure, and enforces **active zero-tolerance safeguards** that suppress unverified figures when source filing disclosures omit primary financial statements.

Technical Architecture & Node Pipeline



Self-Correcting Financial RAG Agent

Core Architecture Components

- **Upstream Guardrail (validate_and_suggest_ticker):**

Pulls the live SEC company directory (~10,000+ active entities) directly into a 24-hour cached registry. Uses rapid fuzzy token matching (rapidfuzz) on tickers and company names combined with LLM semantic fallback to correct misspelled symbols or resolve corporate titles to standard exchange tickers before initiating graph execution.

- **Retrieval Node (retrieve_node):**

Downloads Form 10-K filings directly from SEC EDGAR with compliance User-Agent headers. Combines targeted user inquiry queries with a hardcoded baseline anchor chunk query targeting the *Consolidated Statements of Operations* (net sales, operating income, net income) to guarantee foundational financial context. Chunks are deduplicated while preserving contextual document order.

- **Document Grader (grade_documents_node):**

Employs Pydantic structured output (GradeResult) to determine whether extracted text contains verifiable numerical metrics or merely regulatory disclaimers/exhibits. Acts as a conditional edge router: returns yes to proceed to generation, or no to trigger query rewriting.

- **Query Rewriter (rewrite_query_node):**

If the Grader flags missing financial metrics, the query is rewritten into 3–5 standardized GAAP accounting search keywords (e.g., Net Sales, Revenue, Operating Income) and re-routed to the Retrieval Node. A loop limit ($\text{loop_count} < 3$) serves as a strict circuit breaker against infinite cycles.

- **Generation Node (generate_node):**

Synthesizes verified filing context into an executive-ready dashboard: High-level summary, standardized multi-year Markdown comparison table, and strategic segment highlights. System prompts strictly command the model to cite only verified figures and reject ungrounded speculation.

- **Hallucination Checker (check_hallucination_node):**

A post-generation verification step that validates every figure in the executive summary against the source filing chunks. Returns passed or fail to govern iterative refinement or terminal delivery.

Self-Correcting Financial RAG Agent

- **Integrity Guardrail & Safety Intercept Layer (UI Presentation):**

If regulatory exhibits (such as Item 9A disclosures or Section 302 certifications) fail to provide primary financial statements, the system activates an explicit **Zero-Tolerance Safeguard**. It intercepts missing ground truth, alerts the user that ungrounded hallucination was actively blocked, and marks unverified metrics as *Not Provided* to eliminate commercial pricing, budgeting, or bidding errors.

Key Competencies Demonstrated

- **Orchestration:** Advanced state management with langgraph.graph.StateGraph utilizing cyclical graphs, bounded loop breakers, and conditional edge routing.
- **Integrity & Risk Mitigation:** Built-in zero-hallucination intercept layer that treats missing ground truth as a defensive safeguard rather than a failure state.
- **Cost & Latency Control:** Targeted dual-query retrieval, cached SEC directory lookups, bounded retry loops, and rapid execution using Google Gemini 3.6 Flash.
- **Data Engineering:** Automated SEC EDGAR ingestion, robust fuzzy-to-semantic symbol validation, and schema enforcement via Pydantic.